DEPARTMENT OF MATHEMATICS & COMPUTER SCIENCE

Faculty of Science and Technology · National University of Lesotho

CS3520

Computer Organisation and Architecture I

Week 1 — Computer Abstractions & the Compilation Path

Mr K. O. Setetemela

Course Coordinator · Department of Mathematics & Computer Science

Academic Year 2026/2027 · Semester A

THIS WEEK

What We'll Cover

1 **Where This Course Fits**

How CS3520 connects to your programme and prior courses

2 **Architecture vs Organisation**

Two words we will use precisely, all semester

3 **Eight Great Ideas**

The recurring themes of computer architecture

4 **Layers of Abstraction**

From your C program down to hardware

5 **The Compilation Path**

How code becomes bits the processor executes

6 **Why RISC-V**

The open architecture this course is built on

COURSE CONTEXT

Where CS3520 Fits

CS3520 is the core computer-systems course for B.Eng., B.Sc. Computer Science, B.Sc. Information Systems and B.Sc. Double Major — and the principal carrier of the ACM/IEEE CE2016 Computer Architecture & Organisation knowledge area.

Built On

- CS2420 Digital Logic Design
- CS2431 Programming Fundamentals

→

CS3520 (You Are Here)

- Computer Organisation
- and Architecture I

→

Leads Into

- Operating Systems
- Adv. Computer Architecture
- Embedded Systems
- Final-Year Project

FOUNDATIONAL VOCABULARY

Architecture vs Organisation

Computer Architecture

“What the programmer sees”

Attributes visible to the programmer — the instruction set, data types, addressing modes, and how instructions affect registers and memory.

Architectural question: should this ISA include a multiply instruction?

Computer Organisation

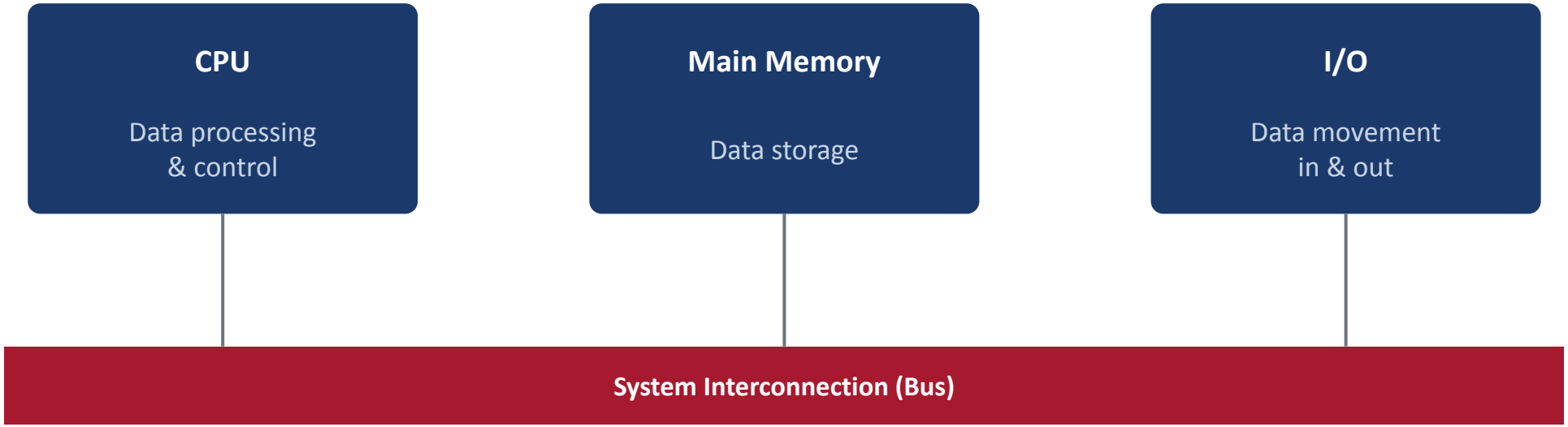
“How it is actually built”

The operational units and interconnections that realise the architecture — control signals, datapaths, and memory technology.

Organisational question: is that multiply built from a dedicated unit, or repeated addition?

Anatomy of a Computer System

Any computer can be described by its structure (how parts connect) and its function (what each part does).



Structure — CPU, memory, I/O and the interconnection between them.

Function — processing, storage, movement, and control of data.

RECURRING THEMES

Eight Great Ideas in Computer Architecture



Design for Moore’s Law



Abstraction to Simplify Design



Make the Common Case Fast



Performance via Parallelism



Performance via Pipelining



Performance via Prediction



Hierarchy of Memories

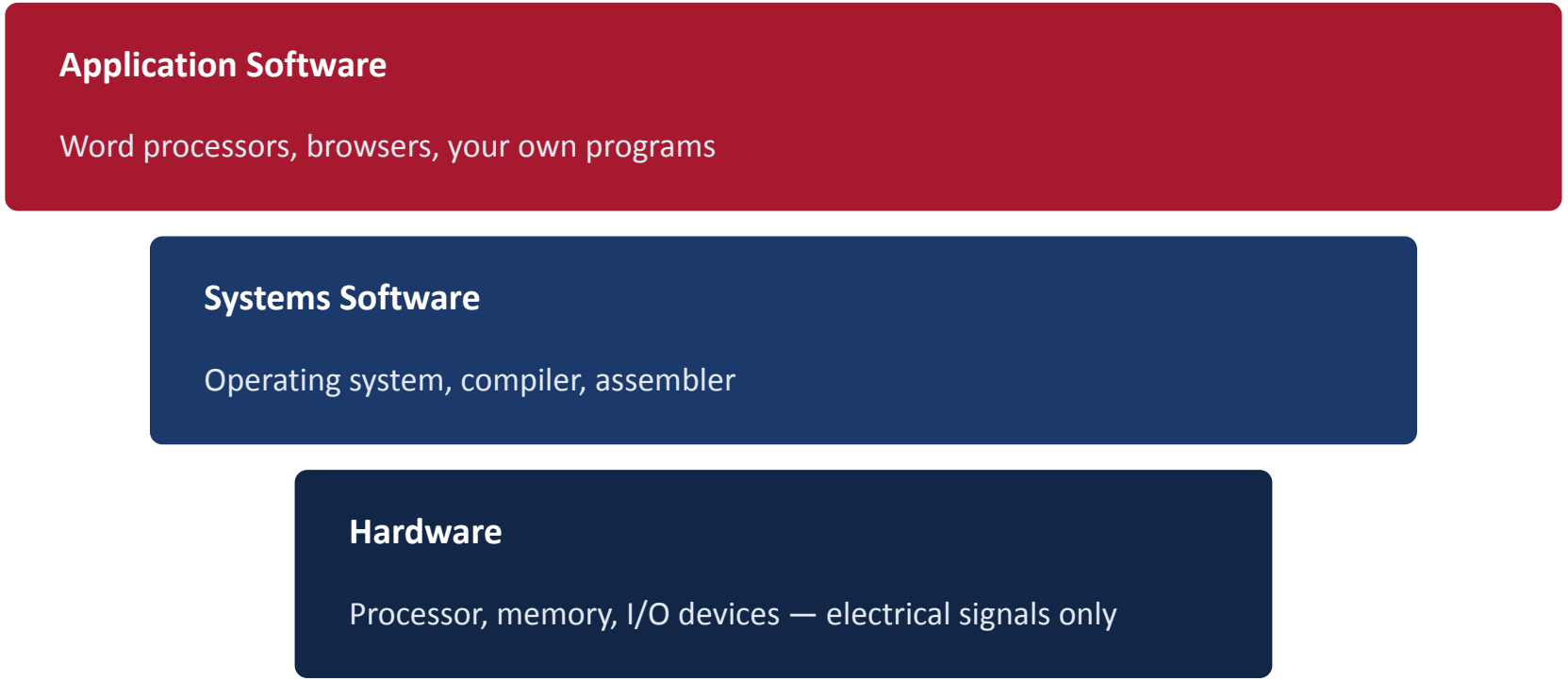


Dependability via Redundancy

BELOW YOUR PROGRAM

Layers of Abstraction

Abstraction — one of our eight great ideas — in action: each layer hides the complexity below it.



Two kinds of systems software matter most: the operating system (manages I/O, memory, sharing) and the compiler (translates your code).

FROM CODE TO BITS

The Compilation Path



Every high-level statement compiles to assembly, then assembles to the 1s and 0s the processor actually executes — all three represent the same computation.

OUR TEACHING ARCHITECTURE

Why This Course Uses RISC-V



Open & Royalty-Free

No licence fees — the full instruction set is public, which matters for a cost-sensitive teaching environment.



Real Industry Momentum

From embedded devices to emerging AI accelerators — RISC-V skills transfer directly to industry.



Free, Open Tools

RARS and Ripes — no expensive toolchains. Install once, use all semester.

You will write and trace RISC-V assembly this month, then build a RISC-V datapath in hardware later this semester.

THINK ABOUT IT

A word processor may contain millions of lines of code — yet the processor beneath it only understands additions, loads, and branches.

Where does all that complexity go? And what would happen if every programmer had to write in binary?

Discuss with a neighbour for two minutes before we continue.

HANDS-ON THIS WEEK

Lab 0 — Environment Setup

Install RARS

RISC-V Assembler and Runtime Simulator — write, assemble and run your first program.

Install Ripes

A visual RISC-V pipeline simulator we'll rely on from Week 6 onward.

Tools Induction

Guided walkthrough: edit → assemble → run → inspect registers.

LAB GROUPS · ICT LAB

Group A	Wed 08:00–10:00
Group B	Fri 08:00–10:00
Group C	Fri 11:10–13:10

Bring a laptop if you have one — ICT Lab machines are available for everyone else.

CHECKPOINT · CLO1

Key Takeaways

1. Distinguish computer architecture (what the programmer sees) from computer organisation (how it is built).

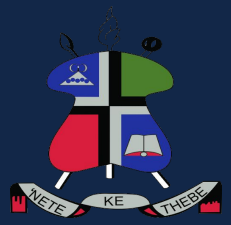
2. Describe a computer system in terms of structure (CPU, memory, I/O, bus) and function (process, store, move, control data).

3. Explain how layers of abstraction let a simple application sit on top of enormously complex hardware.

4. Trace a program from high-level source, through assembly, to the binary instructions a processor executes.



Essential reading: Patterson & Hennessy, Ch. 1 (§1.1–§1.4) • Stallings, Ch. 1 (§1.1–§1.2)



QUESTIONS?

COMING UP — WEEK 2

Measuring & Analysing Computer Performance

Response time, CPI, benchmarking & Amdahl's Law

Mr K. O. Setetemela • ko.setetemela@nul.ls